

Moving away from x86 linux user space



what?

- Other architectures and modes of operation
- Emulating and debugging a whole system, opposed to a single userspace program
- How to approach something low level and unfamiliar

what what?

- x86 real mode
- qemu-system + gdbstub
- google, manuals, osdev.org

why?

- Real mode works similarly to modern microcontrollers, it's useful to understand how it works; moreover, its level of privilege is comparable to ring 0 in protected mode (aka kernel privileges)
- Debugging highly privileged contexts is harder than doing the same for userspace programs, after all there is no kernel that acts as a steward between debugger and debuggee.
And we want to pwn stuff, so we have to do it anyway. Emulation is the easy way to do it, maybe we'll see jtag/swd next year

x86 real mode basics

- Real -> means real addressing mode, there is no virtual memory
 - accessing the address `0x0` means actually accessing address `0x0` in physical memory
- Default operand size is 16 bits
 - Though more memory can be accessed, using segment registers (cs, ss, ds, ...)*
- In systems with a BIOS, we are able to access BIOS functions
 - Kinda like a syscall, they're meant to provide basic input/output functionality until a real kernel can take over
- No BIOS, no fun
 - Without access to BIOS services, everything has to be done from scratch
 - If you want to access hard drives, you better start writing an ATA driver

real mode memory segments

Having to work with only 16 bits has very restrictive limits in terms of memory: the maximum address is 0xffff, which means only 64kb of memory would be available.

Segment registers have been used as a workaround.

Every memory access resolves a pair of [segm_register:address] to a linear (physical) address, by multiplying the segment register by 0x10 and adding it to the address.

This means we can use more memory, but also, that different logical addresses can alias to the same linear address.

Example:

cs = 0x07c0
ip = 0x0000
-----> 0x7c00

cs = 0x0000
ip = 0x7c00
-----> 0x7c00

cs = 0x0700
ip = 0x0c00
-----> 0x7c00

real mode but with more pwn

Real addressing mode

- No page tables means no memory protection!

Every piece of memory is good for a shellcode, it's like the olden days :D

- No page tables also mean stack, code and data share* the a contiguous address space

If we manage to make the stack grow a lot, we might be able to overwrite code or important data!

Moreover, addresses are deterministic more often than not in real mode, so most of the time we can just hardcode stuff, leaks are rarely required.

* segment registers can be used to limit the scope of some memory accesses, but in general, every instruction has access to the whole memory.

real mode but with more pwn

Everything is fair in love, war, and real mode

In ring 0 there is something akin to signals (SIGSEGV, SIGFPE, etc.) to indicate faults: exceptions. An exception causes an interrupt, and interrupts are handled using an IDT Interrupt Descriptor Table.

In real mode this for some reason is called IVT (V for Vector), and its location is by default at physical address 0x0000 to 0x03ff, and contains 256 far* pointers: if we can overwrite that with arbitrary data, we can redirect interrupts!

Usually once out of real mode, the IDT gets moved somewhere else by the kernel, but the same thing still applies. (the format differs)

More reading: https://wiki.osdev.org/Interrupt_Descriptor_Table

* a far pointer includes a segment selector, so when an interrupt happens, the `cs` register can change.

exceptionhandler? i hardly knew her

Most of the IVT entries at boot time are set by SeaBIOS (the BIOS implementation QEMU uses) to F000:FF53, an empty routine which just calls `iret` to immediately return to the code.

This means that everything is fair, divisions by zero won't crash the program, undefined instructions won't either.

If a shellcode is **somewhere** in memory, and if we can manage to send execution flow **somewhere** outside of an infinite loop, there is a good chance that eventually our code will be executed*

* no guarantees about memory state, registers, or shellcode integrity.

```
real-mode-gdb> x/100wx 0x0
0x0: 0xf000ff53 0xf000ff53 0xf000e2c3 0xf000ff53
0x10: 0xf000ff53 0xf000ff54 0xf000ff53 0xf000ff53
0x20: 0xf000fea5 0xf000e987 0xf000d93d 0xf000d93d
0x30: 0xf000d93d 0xf000d93d 0xf000ef57 0xf000d93d
0x40: 0xc0004f12 0xf000f84d 0xf000f841 0xf000e3fe
0x50: 0xf000e739 0xf000f859 0xf000e82e 0xf000efd2
0x60: 0xf000d962 0xf000e6f2 0xf000e6e6 0xf000ff53
0x70: 0xf000ff53 0xf000ff53 0xf00072dc 0xc00089a0
0x80: 0xf000ff53 0xf000ff53 0xf000ff53 0xf000ff53
0x90: 0xf000ff53 0xf000ff53 0xf000ff53 0xf000ff53
0xa0: 0xf000ff53 0xf000ff53 0xf000ff53 0xf000ff53
0xb0: 0xf000ff53 0xf000ff53 0xf000ff53 0xf000ff53
0xc0: 0xf000ff53 0xf000ff53 0xf000ff53 0xf000ff53
0xd0: 0xf000ff53 0xf000ff53 0xf000ff53 0xf000ff53
0xe0: 0xf000ff53 0xf000ff53 0xf000ff53 0xf000ff53
0xf0: 0xf000ff53 0xf000ff53 0xf000ff53 0xf000ff53
0x100: 0xf000ec59 0x9fc0003d 0xf000ff53 0xc00067a0
0x110: 0xf000ff53 0xf000ff53 0xf000ff53 0xf000ff53
0x120: 0xf000ff53 0xf000ff53 0xf000ff53 0xf000ff53
0x130: 0xf000ff53 0xf000ff53 0xf000ff53 0xf000ff53
0x140: 0xf000ff53 0xf000ff53 0xf000ff53 0xf000ff53
0x150: 0xf000ff53 0xf000ff53 0xf000ff53 0xf000ff53
0x160: 0xf000ff53 0xf000ff53 0xf000ff53 0xf000ff53
0x170: 0xf000ff53 0xf000ff53 0xf000ff53 0xf000ff53
0x180: 0x00000000 0x00000000 0x00000000 0x00000000
real-mode-gdb> x/10i 0xffff53
0xffff53: iret
0xffff54: pushd 0xf0a2
0xffff5a: jmp 0xfd86e
0xffff5d: push edi
0xffff5f: push esi
0xffff61: push ebx
0xffff63: mov ecx, eax
0xffff66: mov ebx, edx
0xffff69: test al, 0x7
0xffff6b: jne 0xffffa2
```

real mode with pwn but with some caveats

BIOS and the Stack

BIOS code can actually have a noticeable effect on the state of the program, as it doesn't keep an independent stack register.

This can cause issues when we're aiming to exploit a stack overflow vulnerability, as memory over the top of the stack might not always be controlled by us, and can be affected by the handling of external interrupts (like a timer)

To avoid issues, we might have to optimize our stack overflow exploits to minimize the exposure to external factors. (you have to have blazing fast i/o)



what's next?

- Protected/Long mode (modern kernels)
 - GDT / segmentation (obsolete)
 - Memory paging
- Virtualization Extensions
 - You'll lose hours on end on Intel/AMD manuals
- System Management Mode (look out for this @ corCTF)
- Something else other than x86